MONASH University
Engineering

ECE2071 Project S1 2026 - Week 6 Activity (Malaysia Campus)

Task 3. Acquiring Audio Signals using the built-in Analog to Digital Converter (ADC) on the Microcontroller and Transmission of the ADC values using UART

Synopsis

In this task, we will utilize the in-built analog to digital converter (ADC) on the STM32 Nucleo-L432KC board to acquire audio signals from the audio jack of your computer as analog voltages. Each analog voltage reading will then be transmitted using UART as sequences of 2 bytes (16-bits) to the command line interface coded in Python. The Python script will then write the sequence of bytes representing the raw ADC values into a binary file for further processing.

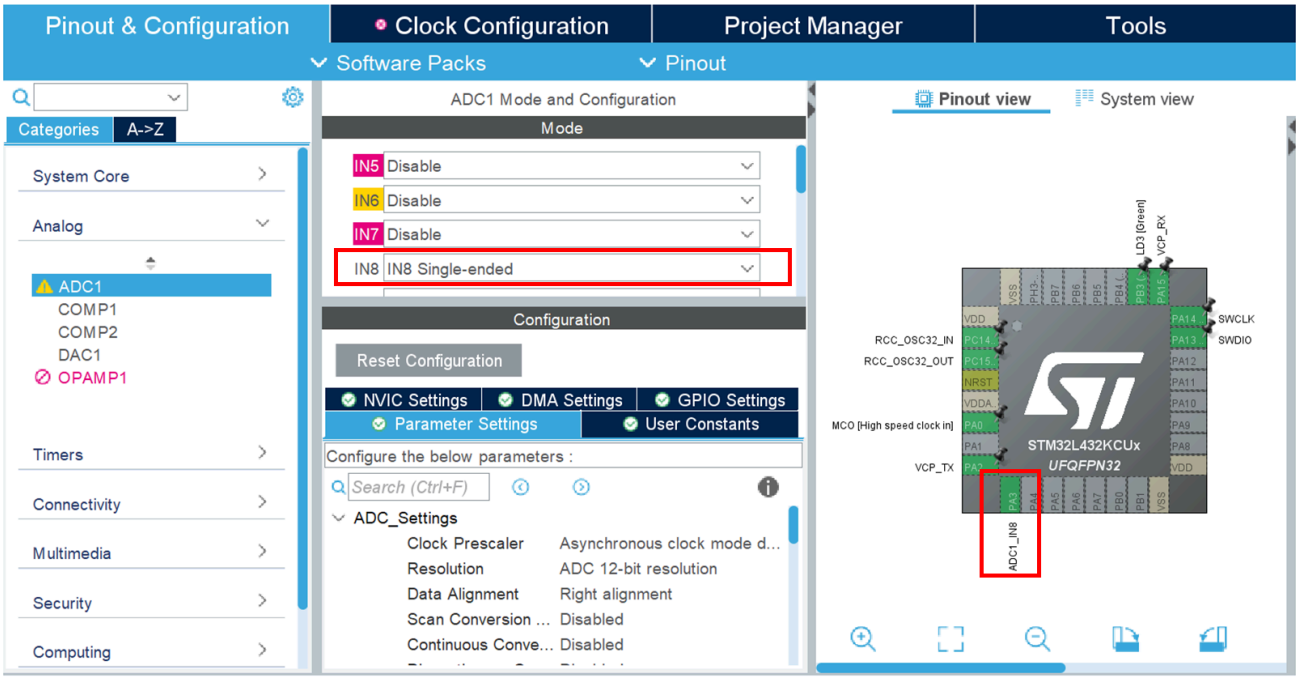
Required Materials

- STM32 Nucleo-L432KC board
- USB Type A-to-Micro B cable
- Personal Computer installed with STM32 CubIDE software, Termite RS-232 terminal and Python Interpreter
- TRRS audio breakout board
- Connecting wires
- Potentiometer
- 2x 1 kΩ resistor
- 1 µF capacitor

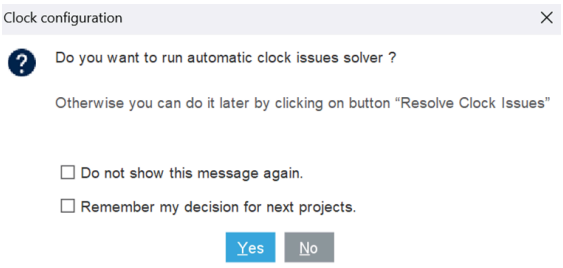
Task 3(a): Acquiring audio signals as analog voltages using the built-in ADC of the microcontroller

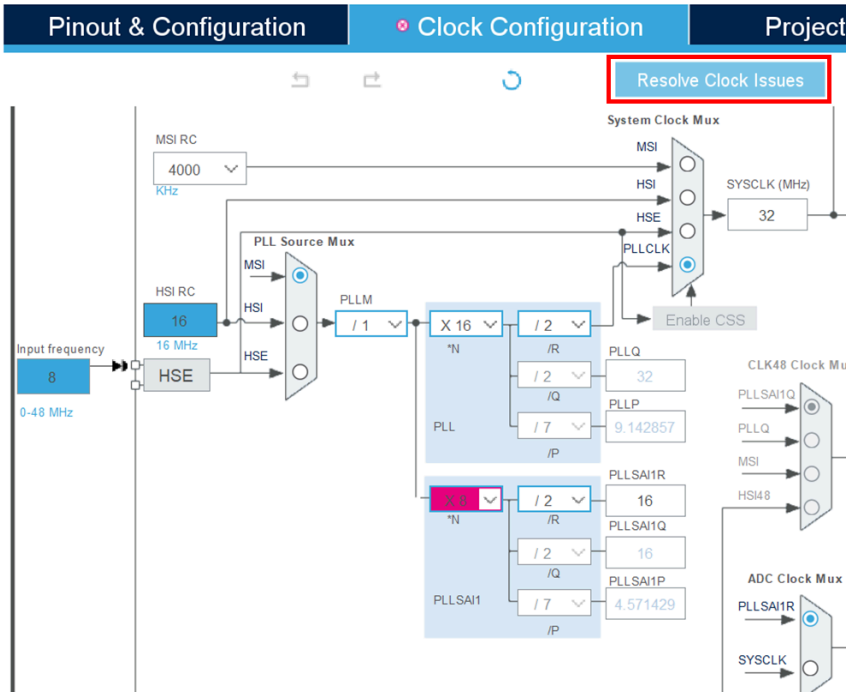
Electrical signals in the real world are continuous and can assume any value that depends on the time at which the signal is recorded. However, for a microcontroller to record and analyze analog voltages, these signals should be converted to discrete digital values using an analog-to-digital converter (ADC). The STM32 Nucleo-L432KC board has an inbuilt ADC named ADC1, which we can use in our project to record any desired analog voltages. In order to record an analog voltage (restricted to within 0 - 3.3V), the ADC1 peripheral on the microcontroller converts each sample of the analog signal to a digital value in between 0 - 4095 (assuming the default 12-bit resolution).

Step 1: Create a new STM32 project on the STM32Cube IDE. In the .ioc file of the project, under the “Pinout & Configuration” tab and then Analog options, click on ADC1. Configure the IN8 input of ADC1 as “IN8 Single-ended” as shown in the image on the next page. This will auto-allocate Pin PA3 of the microcontroller as the analog input to the ADC. Further, the single-ended option will allow measurement of an analog voltage connected to Pin A3 with respect to ground and its conversion to a digital value between 0 - 4095 (based on the default 12-bit ADC resolution). It could also be noted from the ADC settings on the next page that the default ADC data alignment option is set as “Right alignment”, which means that if a 16-bit unsigned integer variable is used to represent the 12-bit ADC value, the rightmost 12-bits of that variable in memory will be used to represent the data stored in the variable.

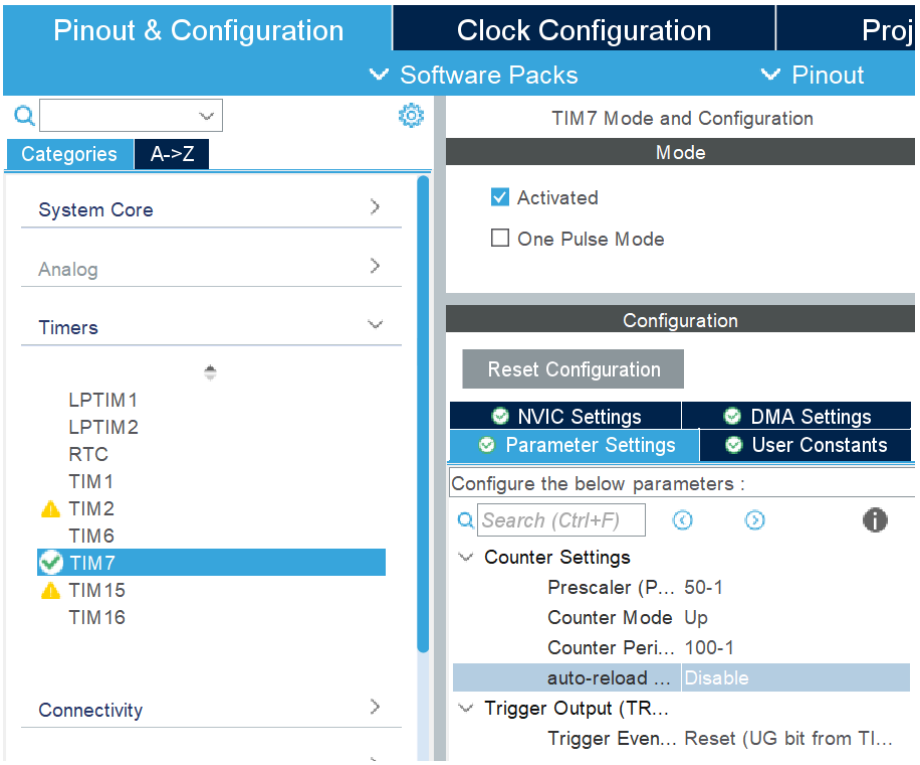


Click on the "Clock Configuration" tab and then click resolve the ADC clock issue by selecting "Yes" on the pop-up window and "Resolve Clock Issues" under the "Clock Configuration" tab as shown in the images below. This should set a clock frequency of 32 MHz for ADC1.

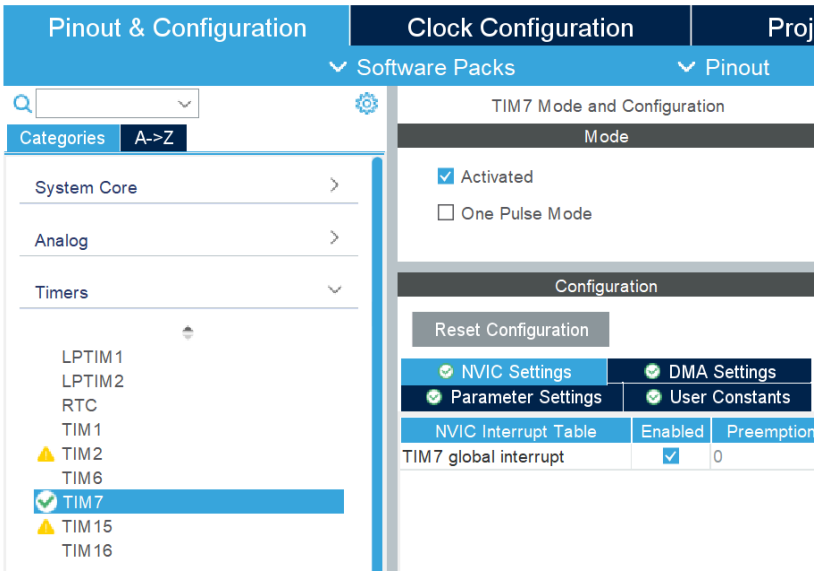




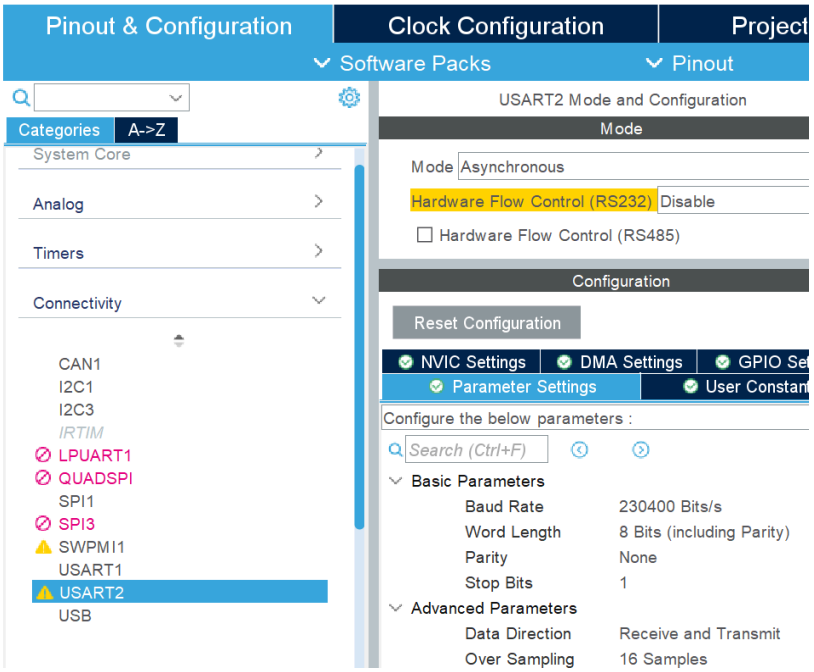
Step 2: Activate and configure timer TIM7 in the .ioc file of the project such that it allows to sample the output of ADC1 at the rate of 6.4 kilo samples/sec. This will be performed by setting the frequency (1/timer_period) of timer TIM7 to 6.4 kHz by choosing the timer parameters as shown in the image below (assuming APB1 clock frequency of 32 MHz).



Then, under NVIC Settings for TIM7, enable global interrupt for TIM7.



Step 3: In the .ioc file of the project, under Connectivity options, select USART2 and then set the Baud rate to 230400 bits/sec while the UART packet format is maintained as 8N1 as shown in the image below. We have chosen a relatively high baud rate for the UART interface in order to minimize any delays in transmission of the ADC output values (sampled at a frequency of 6.4 kS/s) to the command line interface coded in Python. Save the .ioc file and then, generate code.



Step 4: In the `main.c` file, include the `stdio.h` and `string.h` header files in between the **`/* USER CODE BEGIN Includes */`** and **`/* USER CODE END Includes */`** blocks as shown in the image on the next page.

```
/* Private includes ----------
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */
```

Add the following code in between the **`/* USER CODE BEGIN PV */`** and **`/* USER CODE END PV */`** blocks in the main.c file of the project. The 16-bit unsigned integer variable `raw_ADC_value` will be used to temporarily store the result of the latest ADC conversion, which will then be broken down into 2 bytes and stored in the 8-bit unsigned integer array `raw_ADC_value_1`. `temp` is a temporary variable to assist breaking of the 16-bit ADC value into 2 bytes using bitwise manipulation. Further, the character array `string_1` will be used to write the value of the variable `raw_ADC_value` to Termite as a string.

```
/* USER CODE BEGIN PV */
uint16_t raw_ADC_value = 0;
uint8_t raw_ADC_value_1[2] = {0};
uint16_t temp = 0;
char string_1[40] = "\0";
/* USER CODE END PV */
```

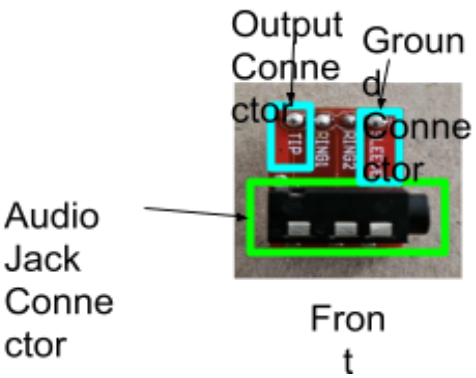
Add the following code in between the **/* USER CODE BEGIN 2 */** and **/* USER CODE END 2 */** blocks in the main.c file of the project to start Timer TIM7 under the interrupt mode.

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim7);
/* USER CODE END 2 */
```

Add the following code in between the **/* USER CODE BEGIN 4 */** and **/* USER CODE END 4 */** blocks in the main.c file of the project to perform one ADC conversion during each period of timer TIM7. Then, save the main.c file.

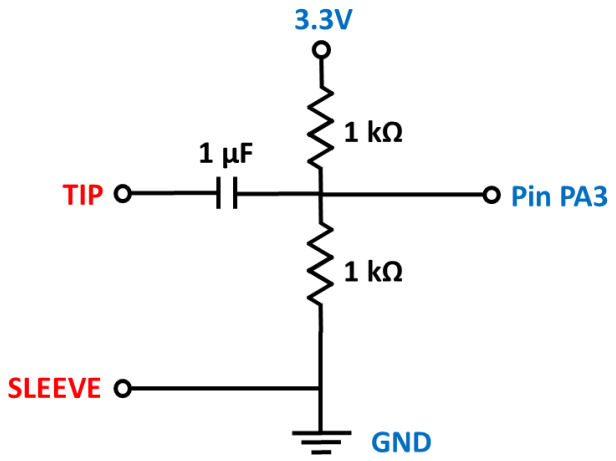
```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback (TIM_HandleTypeDef* htim)
{
    if (htim == &htim7) //If the Callback was executed for ADC timer TIM7
    {
        HAL_ADC_Start(&hadc1); //Start ADC
        HAL_ADC_PollForConversion(&hadc1, 10); //Wait for conversion
        raw_ADC_value = HAL_ADC_GetValue(&hadc1); //Get the unsigned 12-bit ADC data
        //Print raw_ADC_value into string_1
        sprintf(string_1,"raw ADC value: %d\n", raw_ADC_value);
        //Transmit string_1 over UART
        HAL_UART_Transmit(&huart2, (uint8_t*)string_1, strlen(string_1), 10);
        HAL_ADC_Stop(&hadc1); //Stop ADC
    }
}
/* USER CODE END 4 */
```

Step 5: Connect the audio jack connector of TRRS breakout board to the audio jack of your computer using the 3.5mm AUX audio cable provided in the kit. The TRRS breakout board has 4 pins: SLEEVE which is the ground pin, TIP for left audio channel, RING1 for right audio channel and RING2 for microphone input, as shown in the image below.

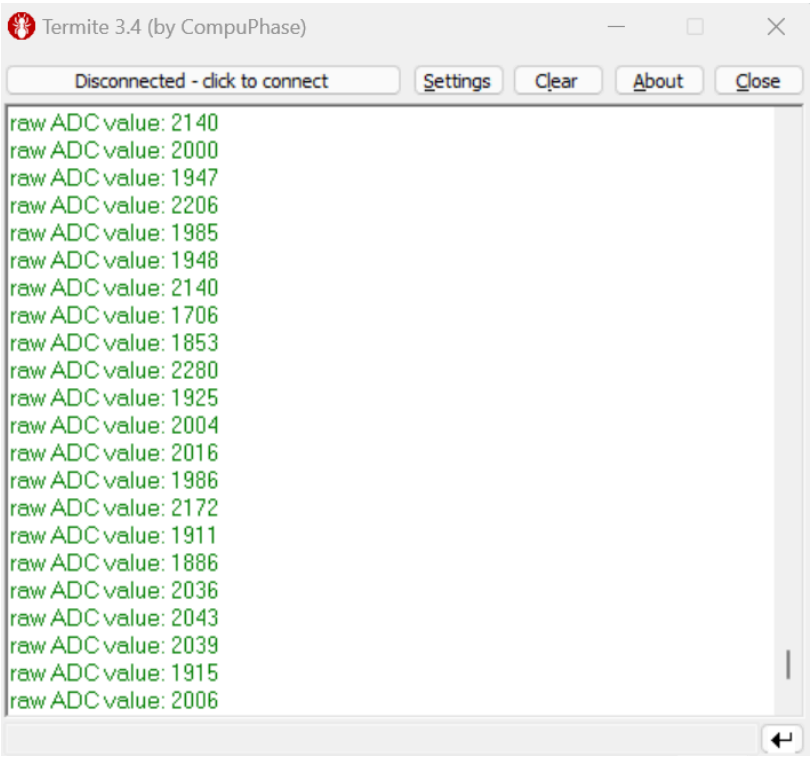


Since the key input used in the project is the audio jack on the TRRS breakout board, the monophonic audio signal from the audio jack of the TRRS breakout board can be acquired through the TIP and SLEEVE pins of the TRRS breakout board. In order to ensure that only positive analog voltages are supplied to the ADC input of the microcontroller, the voltage on the TIP pin should level shifted to above 0V at all times using the level-shifter circuit shown on the next page. The pins indicated in red font in the circuit refer to those on the

TRRS breakout board and the pins indicated in blue font in the circuit refer to those on the STM32 Nucleo-L432KC board. Construct this electrical circuit, and then connect the STM32 Nucleo-L432KC board to your computer using the USB Type A-to-Micro B cable provided in the kit.



Step 6: Set the audio playback volume on your laptop to high volume and then play your favourite song in the background. Build the STM32 project on the IDE and then run as STM32 C/C++ application. Open Termite RS-232 terminal on the computer and ensure that the COM port of the computer connected to the STM32 Nucleo-L432KC board is selected. Further, set the baudrate to 230400 bits/second and maintain the UART packet format as 8N1 on Termite. You should now be able view the ADC output values that represent the audio playing on the computer as shown in the figure below. Since the ADC output values are sent to Termite at a relatively fast frequency of 6.4 kHz, disconnect Termite once the values start appearing in order to prevent Termite from crashing.



Task 3(b): Transmitting the ADC output values as 2 bytes to the Command Line Interface and saving the data as a binary file for further processing

Step 7: Modify the `HAL_TIM_PeriodElapsedCallback()` function in the main.c file as shown in the image below to break the ADC output into 2 bytes through bitwise manipulation, which are then transmitted over the UART interface.

```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback (TIM_HandleTypeDef* htim)
{
    if (htim == &htim7) //If the Callback was executed for ADC timer TIM7
    {
        HAL_ADC_Start(&hadc1); //Start ADC
        HAL_ADC_PollForConversion(&hadc1, 10); //Wait for conversion
        raw_ADC_value = HAL_ADC_GetValue(&hadc1); //Get the unsigned 12-bit ADC data
        //Print raw_ADC_value into string_1
        //sprintf(string_1,"raw ADC value: %d\n", raw_ADC_value);
        //Transmit string_1 over UART
        //HAL_UART_Transmit(&huart2, (uint8_t*)string_1, strlen(string_1), 10);
        //Extract least significant 8 bits of ADC data
        temp = raw_ADC_value & 0xFF;
        raw_ADC_value_1[0] = temp;
        //Extract most significant 8 bits of ADC data
        temp = raw_ADC_value >> 8;
        raw_ADC_value_1[1] = temp;
        //Transmit 16-bit ADC data through UART
        HAL_UART_Transmit(&huart2, (uint8_t*)raw_ADC_value_1, 2, 10);
        HAL_ADC_Stop(&hadc1); //Stop ADC
    }
}
/* USER CODE END 4 */
```

Step 8: Create a Python script named read_serial_1.py containing the following code on your computer. The Python script reads 5000 bytes of ADC data transmitted by the STM32 Nucleo-L432KC board, equivalent to 2500 ADC conversions, and writes the data into a binary file “raw_ADC_values.data”.

read_serial_1.py ×

C: > code > read_serial_1.py > ...

```python
import serial
#set serial UART parameters (note that baudrate should be higher than ADC sampling rate)
ser = serial.Serial(port = "COM5", baudrate = 230400, bytesize = 8, parity="N", stopbits = 1, timeout = 5)
#check whether the serial communication is active
print(ser.name)
#open the binary file to write the raw ADC values
file_1 = open("raw_ADC_values.data","ab")
x = ser.read(5000) #read 5000 bytes over UART
file_1.write(x) #write 5000 bytes to file
file_1.close()
ser.close()
```

Step 9: Again, set the audio playback volume on your laptop to about 70% and then play your favourite song in the background. Build the modified STM32 project on the IDE and then run as STM32 C/C++ application. On the powershell, navigate to the folder where the Python script read_serial_1.py has been saved, and then run the Python script as shown in the image below. The binary file “raw_ADC_values.data” should now be stored on your computer in the same folder as the Python script. You may open the binary file on any text editor to view its contents. Note that the data is stored as bytes rather than ASCII characters.

```
Windows PowerShell
PS C:\code> python read_serial_1.py
COM5
PS C:\code>
```

Modify the STM32 project and the command line interface coded in Python, and write a C processing program in order to incorporate the following functionalities towards the project specifications in the unit.

1. Modify the Python script such that it could read 50 kilobytes of ADC output that represents the song playing on the computer. This could be done by repeatedly reading the ADC output in smaller chunks of 500 bytes each and then storing each chunk of data immediately after its read into a binary file.
2. Write a C processing program in VS Code connected to WSL in order to read the binary file containing the ADC output and utilize the data in the binary file to generate a .WAV file that could be used to play the recorded audio. As of now, the binary file could be copied into a location on the computer accessible by the C program within the WSL environment before running the program. The major difference between the binary file containing the ADC output and the generated .WAV file are summarized as follows:
    - The .WAV file contains 44 bytes of **WAV header information** that precedes the ADC output in the file. The WAV header is what allows the .WAV file to be recognized as an audio file on a computer and allows its playback. Note that **bytes used to represent numbers in the WAV header follow the *little endian* format**.
    - The ADC data in the binary file that currently ranges from 0 - 4095 should be scaled within the range of a 16-bit signed integer (-32768 to +32767) before being entered into the .WAV file. Note that **bytes used to represent the scaled ADC data should also follow the *little endian* format**.

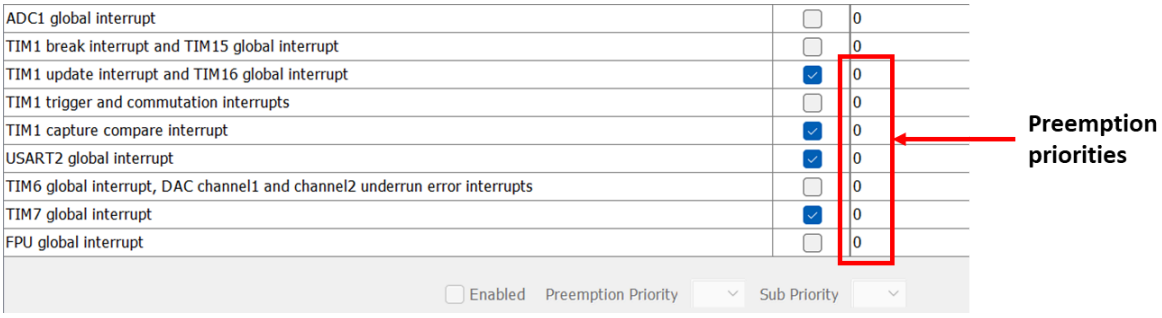
The format of the .WAV file (adapted from https://docs.fileformat.com/audio/wav/) is as follows.

Byte Locations	Description
0 - 3	Indicates the RIFF file type. Should be saved as **"RIFF"**. Each byte is the associated capitalized character (eg. byte 1 → R = ASCII value of 82 or 0x52)
4 - 7	The **total size of the file in bytes including the 44 byte header** = 50000 + 44 in the present case. Value is saved over the full 4 bytes (as a 32 bit integer). Follows the *little endian* format.
8 - 11	Indicates the WAV file type. Should be saved as **"WAVE"**. Each byte is the associated capitalized character (eg. byte 1 → W = ASCII value of 87 or 0x57)
12 - 15	Format chunk marker. Should be saved as **"fmt " (including the whitespace)**. Each byte is the associated character (eg. byte 1 → f = ASCII value of 102 or 0x66)
16 - 19	**Length of the format data based on the format chunk marker = 16**. Value is saved over the full 4 bytes (as a 32 bit integer). Follows the *little endian* format.
20 - 21	**Audio format. For a WAV, this is PCM which is represented as 1.** Value is saved over the full 2 bytes (as a 16 bit integer). Follows the *little endian* format.

22 - 23	**Number of audio channels = 1 for monophonic audio.** Value is saved over the full 2 bytes (as a 16 bit integer). Follows the *little endian* format.
24- 27	**Sampling rate of the saved audio (samples/second)** = 6400 samples/second in the present case. Value is saved over the full 4 bytes (as a 32 bit integer). Follows the *little endian* format.
28 - 31	**Bytes per second of the saved audio.** Value is saved over the full 4 bytes (as a 32 bit integer). This can be calculated as (Sample rate * Bits per sample * Number of channels)/8 = (6400*16*1)/8 in the present case). Follows the *little endian* format.
32 - 33	**Bytes per sample per channel of the saved audio.** Value is saved over the full 2 bytes (as a 16 bit integer). This can be calculated as (Bits per sample * Number of channels)/8 = (16*1)/8 in the present case). Follows the *little endian* format.
34 - 35	**Bits per sample** = 16 in the present case. Value is saved over the full 2 bytes (as a 16 bit integer). Follows the *little endian* format.
36 - 39	Data chunk header. Should be saved as **"data"**. Each byte is the associated character (eg. byte 1 = d = ASCII character 100 or 0x64)
40 - 43	**The total size of the DATA in bytes EXCLUDING the 44 byte header** = 50000 in the present case. Value is saved over the full 4 bytes (as a 32 bit integer). Follows the *little endian* format.
44 onward	**ADC data representing the audio signal scaled within the range of a 16-bit signed integer** (-32768 to +32767). Follows the *little endian* format.

3. Integrate the STM32 project constructed at the end of Week 5 lab with this STM32 project and the Python script such that the system now performs actual audio recording either in manual or distance trigger mode based on user input (via the command line interface). Note that this would bring in a situation where there are four interrupts that implements four different functionalities in parallel on the STM32 microcontroller:
   - Timer-based interrupt 1 to trigger the ultrasonic sensor for distance measurement every 500 ms (TIM16 in Task_2)
   - Timer-based interrupt 2 to calculate the distance based on the width of the Echo pulse generated by the ultrasonic sensor (TIM1 in Task_2)
   - Timer-based interrupt 3 to perform ADC conversions at a fixed sampling rate of 6.4kS/s (TIM7 in Task_3)
   - UART-based interrupt that receives characters byte by byte from the command line interface implemented in Python

   In such a situation, as a programmer, you would need to assign preemption priorities to some of these interrupts to have more control on how the system responds to changes in its environment. By default, all interrupts have been assigned a preemption priority of 0 (highest priority) on the STM32 microcontroller. You can view this information by navigating to System Core → NVIC (Nested Vectored Interrupt Controller) in the .ioc file of the project as shown in the figure on the next page.



The NVIC allows prioritization of interrupt handling based on their respective preemption priorities. For instance, the NVIC ensures that an interrupt with higher preemption priority is completed first even if a lower priority interrupt is triggered first and still being processed. In such a situation, the lower priority interrupt is temporarily paused until the higher priority interrupt is completed, after which the lower priority interrupt resumes and tries to complete its activities. Interrupts of the same preemption priority cannot block each other from completion once they have been triggered. On the STM32 microcontroller, 0 is the value assigned for highest preemption priority and 15 is the value set for lowest preemption priority for interrupts. By default, all interrupts are treated with the same and highest priority, and are therefore allocated a preemption priority of 0 by the NVIC. Thus, none of the interrupts block each other's activity, by default, unless the programmer manually assigns distinct preemption priorities to interrupts based on the requirements of the system.